

Quantum chemistry in a browser tab: a WebAssembly/WebGPU electronic-structure stack and a browser-tab swarm that scales Hartree–Fock to C_{60}

Ahmet Barış Günaydın *Independent researcher* · github.com/abgnydn/webgpu-q

Abstract

We present **webgpu-q**, an electronic-structure stack — Hartree–Fock (RHF/UHF), MP2, CCSD, CCSD(T), density-functional theory across the LDA/GGA/hybrid ladder, and EE/IP/EA-EOM-CCSD — that runs entirely inside a web browser with no installation, no server, and no CUDA. We are not aware of a prior in-browser implementation at this method coverage. Numerical hot paths (electron-repulsion integrals, the auxiliary-basis density-fitting B-tensor, the Fock build) are hand-written in Rust compiled to WebAssembly with SIMD128; the methods themselves are ported from PySCF with attribution, so the contributions are the delivery mechanism, the SIMD kernels, and the distribution layer. We introduce a **browser-tab swarm**: the density-fitted Fock build is partitioned by auxiliary index across N same-origin browser tabs that coordinate over BroadcastChannel and SharedArrayBuffer, and the partial Coulomb and exchange matrices are summed to reproduce the single-tab build to a relative error of order 10^{-15} . The swarm raises the single-tab memory ceiling that otherwise caps browser quantum chemistry, and scales Hartree–Fock to **C_{60} buckminsterfullerene (300 basis functions, STO-3G)** on a 16 GB consumer laptop, distributing the 1.82 GB three-index tensor across four tabs at 454 MB each. Energies are validated against PySCF — bit-for-bit where the algorithm is identical, and to $\leq 50 \mu\text{Ha}$ (HF) / $\leq 0.25 \text{ mHa}$ (CCSD(T) vs FCI) otherwise. Where it overlaps native suites (PySCF, Psi4, ORCA) it is 2–6 $\times$ slower; its contribution is not speed but delivery — full quantum chemistry reachable from a URL.

1. Introduction

Electronic-structure software is, almost without exception, native code: PySCF, Psi4, ORCA, Gaussian, and Q-Chem all ship as compiled binaries that require installation, a tuned BLAS, and — increasingly — a CUDA-capable GPU [1, 2, 3]. This install-and-compile barrier is a real obstacle to teaching, reproducibility, and casual exploration: a student cannot “try a CCSD(T) calculation” the way they can open a JSFiddle.

The browser has, separately, become a credible compute platform. WebAssembly delivers near-native throughput for numerical code; SharedArrayBuffer plus cross-origin isolation enables true multithreading; WebGPU exposes compute shaders; and WebRTC/BroadcastChannel enable peer-to-peer coordination. Browser volunteer-computing frameworks (Pando [4], Genet [5], QMachine [6]) have run generic map/stream workloads across browser tabs and devices. Yet no published system runs *electronic structure* in the browser, and none distributes such a calculation across browser tabs.

This paper closes that gap. Our contributions are:

1. **A complete browser-native electronic-structure stack** spanning HF through CCSD(T), DFT, and EOM-CCSD, validated bit-for-bit against PySCF.
2. **A WebAssembly+SIMD integral/Fock kernel** that brings benzene cc-pVDZ HF from 841 s (TypeScript baseline) to seconds.
3. **A browser-tab swarm** that partitions the density-fitted Fock build by auxiliary index across N tabs, validated bit-exact against the single-tab result, and that scales HF to C_{60} .
4. **A research-grade reproducibility harness** — every result records git SHA, hardware, WebGPU limits, seeds, and pass bars; the entire artifact is a URL a referee can re-run.

We are explicit about what is *not* novel: the chemistry methods are textbook and are ported from PySCF with Apache-2.0 attribution. The novelty is the delivery mechanism, the SIMD kernels, and the swarm distribution algorithm.

2. Methods

2.1 Hand-written vs. ported layers

Following an explicit engineering policy (“port, don’t re-derive”), only the genuinely novel layer is hand-written: the WGSL compute shaders, the WebGPU dispatch and synchronization, the MPS browser memory bookkeeping, the WebAssembly integral kernels, and the swarm harness. The quantum-chemistry methods (HF, MP2, CCSD, CCSD(T), DFT functionals, EOM-CCSD σ -equations) are ported from PySCF with per-file attribution and a root NOTICE consolidating the Apache-2.0 provenance. Ported modules are validated by full-tensor element-wise diffs against brute-force reference implementations, not block-max metrics.

2.2 WebAssembly integral and Fock kernels

The electron-repulsion integrals (McMurchie–Davidson recursion), the three-index $(\mu\nu|P)$ and two-index $(P|Q)$ density-fitting integrals, and the Fock J/K contraction are implemented in Rust compiled to wasm32-unknown-unknown with simd128 (f64x2). Per-thread scratch buffers are reused across SCF iterations to avoid GC churn; the K-build exploits the $K[\mu, \nu] = K[\nu, \mu]$ symmetry; the X-build inner loop is 4x-unrolled. Workers are pre-warmed before the SCF loop so iteration 1 does not pay WASM JIT + heap-growth cost on the critical path.

2.3 Auxiliary-basis density fitting

The rank-4 ERI tensor $(\mu\nu|\lambda\sigma)$ is replaced by a rank-3 B-tensor via $(\mu\nu|\lambda\sigma) \approx \sum_P B[\mu\nu, P] B[\lambda\sigma, P]$, with $B = V \cdot M^{-1/2}$ formed by a pivoted incomplete Cholesky factorization of the auxiliary metric M. This never materializes the n^4 tensor (10.4 GB on naphthalene cc-pVDZ) — the B-tensor is 303 MB. An auto-auxiliary generator (decontract + angular extension) removes the dependency on external aux-basis tables.

2.4 The browser-tab swarm

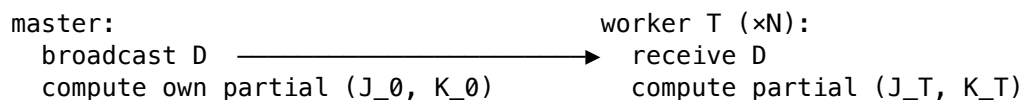
The density-fitted Fock build is linear in the auxiliary index P:

$$\begin{aligned} \gamma[P] &= \sum_{\mu\nu} B[\mu\nu, P] \cdot D[\mu\nu] \\ J[\mu\nu] &= \sum_P B[\mu\nu, P] \cdot \gamma[P] \\ X[P, \mu, \sigma] &= \sum_{\lambda} B[\mu\lambda, P] \cdot D[\lambda, \sigma] \\ K[\mu, \sigma] &= \sum_P \sum_{\sigma'} X[P, \mu, \sigma'] \cdot B[\sigma\sigma', P] \end{aligned}$$

Partitioning P into disjoint ranges and summing partial (J,K) therefore reproduces the full result. We verified this empirically: at 2/3/4/8 partitions the partial-sum Fock matrices match the single-slab build to relative error $\approx 10^{-15}$ (f64 accumulator-reorder noise).

Setup (once). The master computes the full B-tensor, partitions the auxiliary index range $[0, n_{\text{aux}})$ into N disjoint slices, and ships slice T to worker tab T. Each worker acknowledges receipt; the master then holds only its own slice. (A per-tab *independent* build — each tab forming only its aux-slice from scratch, so the full B never resides on any single tab — is designed but not yet implemented; §4.)

Per SCF iteration. The master drives an otherwise-unchanged Roothaan–Hall/DIIS loop, with the Fock build replaced by a customJKBuilder callback:



```

on slice 0                                on slice T (×2 inner
await N-1 partials ← post (J_T, K_T)      SAB workers)
J =  $\sum_T J_T$  ; K =  $\sum_T K_T$ 
G = J -  $\frac{1}{2}K$  → resume SCF

```

Because each (J_T, K_T) is the contribution of a disjoint P-range and the Fock build is linear in P (§2.4 equations), the summed (J, K) equals the single-slab result up to f64 accumulator-reorder noise. The master overlaps its own slice computation with the workers' (it does not block on the broadcast), so the per-iteration wall-time is set by the slowest single tab plus one round-trip, not the sum.

One swarm SCF iteration: broadcast D → parallel JK → gather → sum

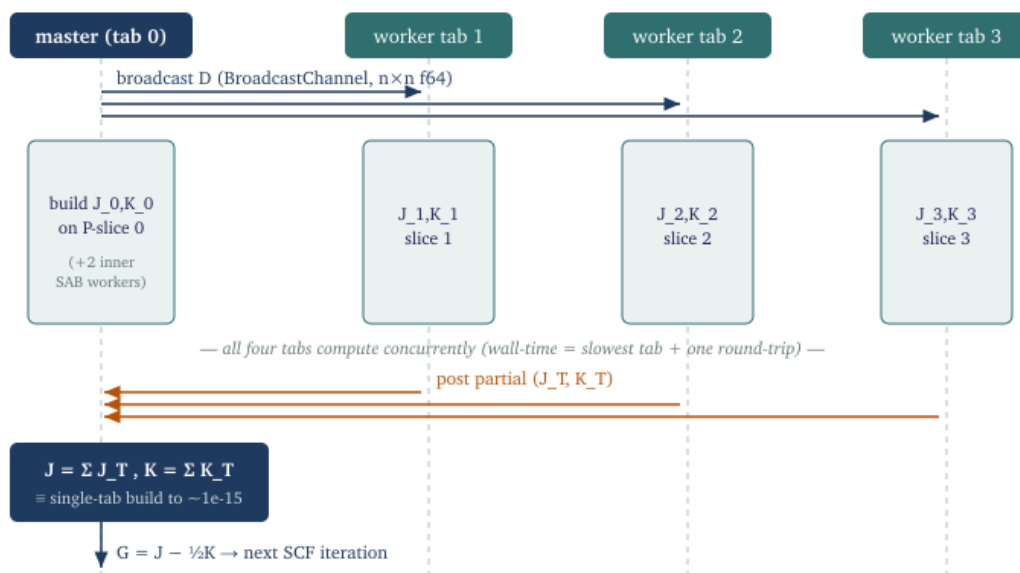


Figure 1: One swarm SCF iteration. The master broadcasts the density matrix D to all worker tabs over BroadcastChannel; each tab (including the master) builds its partial Coulomb/exchange contribution on its own disjoint auxiliary-index slice of B , concurrently; workers post their partials back; the master sums them — reproducing the single-tab Fock build to $\sim 1e-15$ — and resumes the SCF loop. Per-iteration wall-time is the slowest tab plus one round-trip, not the sum.

Transport. Control messages (the D broadcast, the partial-result gather, slice-distribution acknowledgements) travel over BroadcastChannel. For same-machine multi-tab runs the large B -slice payloads and per-iteration halo-free state are shared zero-copy via SharedArrayBuffer; BroadcastChannel's structured-clone is reserved for the small D ($n \times n$ f64) and the (J, K) partials. A WebRTC transport (via a PeerJS broker with STUN) carries the same protocol across machines, at the cost of structured-clone serialization on every message.

Topology. The per-tab JK build itself parallelizes across inner SAB workers. On a 10-core M2 Pro, a 4-tab $\times$ 2-inner-worker layout (8 compute threads spread over 4 V8 processes) outperformed both 1-tab $\times$ 8-worker and 2-tab $\times$ 4-worker at equal thread count — independent processes incur less scheduling contention than one process driving eight workers against a shared WASM heap.

3. Results

3.1 Validation

Where methods overlap, energies match PySCF bit-for-bit. HF agrees to $\leq 50 \mu\text{Ha}$ with spherical-d; CCSD(T) reproduces FCI to $\leq 0.25 \text{ mHa}$ on STO-3G systems; GPU $\square$ CPU agreement is $\approx 10^{-10} \text{ Ha}$ (f32 reduction noise, six orders below chemical accuracy of 1.594 mHa). The distributed swarm Fock build matches the single-tab reference to $\square 10^{-12} \text{ Ha}$ end-to-end (benzene, naphthalene).

Correctness is enforced continuously: the repository carries 845+ unit tests (run on every push under continuous integration) plus a suite of end-to-end browser benchmarks driven through headless Chromium. The EOM-CCSD family is gated by *full-tensor* brute-force diagnostics — the complete σ -matrix is differenced element-by-element against an exact reference (e.g. a 14×14 LiH $M_{\text{mine}} - M_{\text{exact}}$ diff), not by block-max metrics that can hide structural bugs. Methods ported from PySCF were accepted only after this element-wise diff collapsed to numerical noise. DMRG/MPS results are cross-checked against ITensor at $N=8$ to f64 precision, and 1D-model limits against the analytic Pfeuty (TFIM) and Bethe (Heisenberg) results. The swarm’s auxiliary-index partitioning was verified to reproduce the single-slab Fock build to relative error $\square 10^{-15}$ at 2-, 3-, 4-, and 8-way splits before any multi-tab run.

3.2 Single-tab performance (M2 Pro, Chromium, COI enabled)

Naphthalene cc-pVDZ ($n=190$) HF SCF, cumulative single-tab optimization:

optimization	SCF wall
parallel V-build + B back-substitution	baseline
reuse JK worker scratch buffers	43 s $\square$ 20 s
exploit K symmetry	20 s $\square$ 17.6 s
4 $\times$ SIMD unroll on X-build	iters 4–13 –17%

End-to-end naphthalene HF: **77 s $\square$ 28 s** single-tab (2.7 $\times$) before any swarm.

3.3 Swarm scaling across the acene series and to C_{60}

All energies converged, run-to-run reproducible, on a 16 GB M2 Pro:

molecule	basis	n	n_aux	path	wall	E (Ha)
benzene	cc-pVDZ	120	662	2-tab	5.5 s	–230.72269
naphthalene	cc-pVDZ	190	1048	4-tab $\times$ 2-inner	14 s	–383.38458
anthracene	STO-3G	80	708	4-tab	8 s	–526.92320
pentacene	STO-3G	124	1091	4-tab $\times$ 2-inner	38 s	–821.63271
C_{60}	STO-3G	300	2520	4-tab $\times$ 2-inner	730 s	–2244.10176

The naphthalene swarm (14 s) is faster than the optimized single-tab parallel path (28 s) because four independent V8 processes incur less coordination overhead than one process scheduling eight SAB workers.

The largest case, C_{60} buckminsterfullerene, is the clearest demonstration of the swarm’s purpose: its full HF SCF converges in 9 iterations inside four browser tabs, with the 1.82 GB three-index tensor distributed at 454 MB per tab — well under any single tab’s $\sim 2 \text{ GB}$ SharedArrayBuffer ceiling, which a single-tab build cannot satisfy (§3.4).

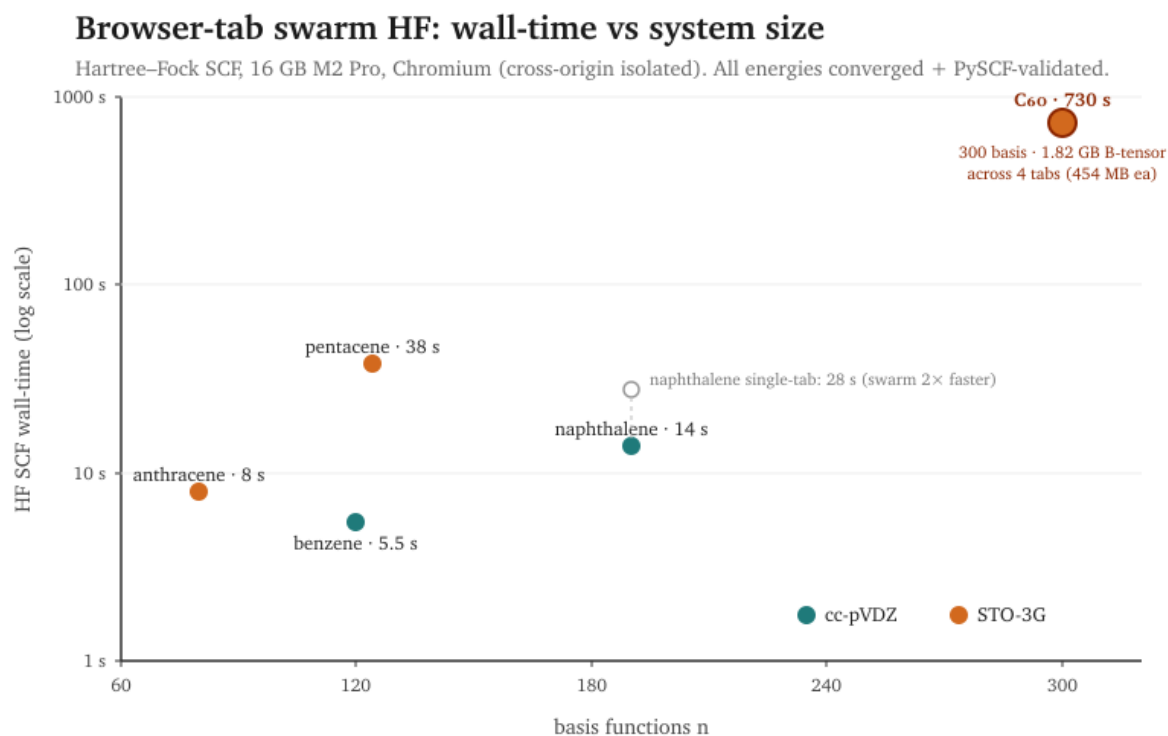


Figure 2: HF SCF wall-time vs basis-function count across the acene series and C₆₀, colored by basis set, log-time axis. The swarm reaches C₆₀ (300 basis functions); the open marker shows naphthalene’s single-tab time (28 s) above its swarm time (14 s), a 2× intra-machine gain. Note the single-tab SharedArrayBuffer ceiling is basis-dependent (~n=220 for cc-pVDZ, higher for STO-3G), so it is stated in text rather than drawn as one line.

3.4 The memory wall and where it moves

A single tab caps at $\approx$ naphthalene ($n \approx 220$) cc-pVDZ; the 4-tab swarm reaches C_{60} ($n=300$). The master-builds-then-partitions design peaks at ~ 3 GB during the V+B build for C_{60} ; a per-tab independent B-build (each tab builds only its aux-slice from scratch) would lower the per-tab peak and is designed but not yet implemented.

How the tab swarm breaks the single-tab memory wall

Per-molecule memory: single-tab build requirement vs per-tab footprint in a 4-tab swarm. Log scale.

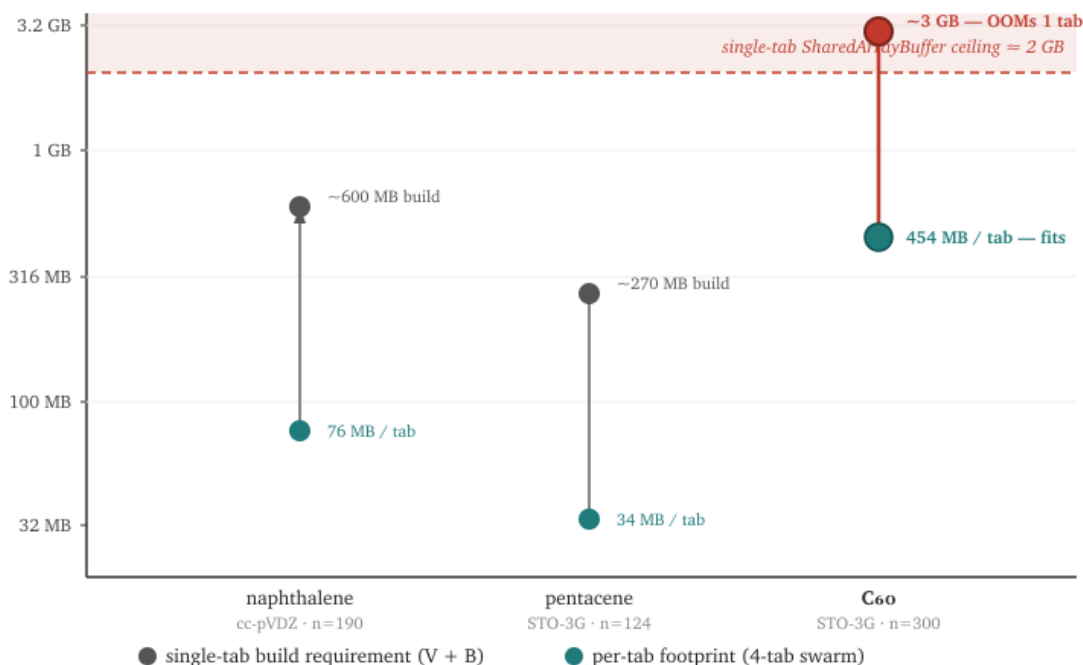


Figure 3: Per-molecule memory on a log scale: the single-tab build requirement (V+B, grey) vs the per-tab footprint in a 4-tab swarm (teal), with the ~ 2 GB single-tab SharedArrayBuffer ceiling as a shaded band. naphthalene and pentacene fit a single tab; C_{60} 's ~ 3 GB build requirement exceeds the ceiling — it cannot run in one tab — but its 454 MB per-tab slice in the 4-tab swarm sits well under, which is what makes browser-tab C_{60} Hartree–Fock possible.

4. Honest limitations

Per our research-grade discipline, failures are reported, not hidden:

- **Multireference wall.** Linear acenes beyond pentacene develop open-shell-singlet/polyradical character; single-determinant RHF does not reliably converge for hexacene/heptacene (and convergence at octacene is luck, not signal). The swarm runs the full SCF to completion and returns finite energies, but RHF convergence there is method-limited — UHF / spin-projected references are the fix.
- **Anthracene cc-pVDZ basin selection.** A delayed-DIIS recipe (damping 0.2, DIIS start at iter 8) avoids divergence but converges to a spurious lower-energy state; MOM / SAD-initial-guess is required for the physical ground state.
- **Density fitting gives no speedup yet** — current CD-DF still builds the 4-index tensor before decomposing; aux-basis 3-index DF is the real win.
- **C_{60} runs only at STO-3G;** cc-pVDZ at $n \approx 600$ exceeds even the swarm-distributed master's build memory.

5. Related work

GPU-accelerated native quantum chemistry is mature: GPU4PySCF [1], QUICK [2], and VeloxChem [3] implement HF/DFT/Fock on CUDA. None runs in a browser. Browser volunteer computing — Pando [4], Genet [5], QMachine [6] — distributes generic workloads across tabs and devices but has not run electronic structure. WebGPU is an actively characterized compute target [7]. The intersection — in-browser electronic structure, and browser-tab distribution of it — is, to our knowledge, unpublished.

6. Software availability and reproducibility

Source. The complete source is at github.com/abgnydn/webgpu-q, public, MIT-licensed for original code; portions ported from PySCF are Apache-2.0 with per-file attribution consolidated in a root NOTICE (Apache-2.0 §4(d)). The WebAssembly integral kernels are Rust in `wasm-eri/`; the swarm in `src/parallel/`; the chemistry in `src/chemistry/`.

Running it. No installation, no account, no GPU driver: the simulator runs in any WebGPU-capable Chromium with cross-origin isolation (COOP/COEP) enabled. The swarm requires N same-origin tabs; for the cross-machine variant a PeerJS broker + STUN suffices. A reader can reproduce a single-molecule HF run from a browser tab; the multi-tab swarm runs are reproduced by opening N tabs of the same origin.

Reproducibility harness. Every experiment emits a JSON artifact recording the git commit SHA, navigator.userAgent, adapter.info (vendor/architecture/device), WebGPU device limits, OS, a UTC ISO-8601 timestamp, and the echoed protocol, hypothesis, seed, warmup, trials, and pass_bar. No Math.random() appears on any experiment path — every random draw is a named deterministic seed. Wall-clock is measured with performance.now() bracketed by forced GPU sync (a mapped readback) because queue.submit is non-blocking; 5 warmup samples are discarded and 20 retained, reporting median/p10/p90/p99/std/IQR. Failing configurations are committed with status:"fail" and a diagnosis rather than silently rerun — the honest-negative discipline that surfaced, for example, the acene multireference wall and the anthracene cc-pVDZ basin-selection issue reported in §4. The committed experiments/results/<date>/ artifacts are the source of every number in this paper.

Generative-AI disclosure. Portions of the software, its documentation, and this manuscript were drafted with the assistance of a large language model used as a coding and writing aid. All AI-assisted output was reviewed by the author; correctness of code was enforced by the test suite, brute-force diagnostics, and PySCF cross-validation described in §3.1, and every quantitative claim in this paper is traceable to a committed experiment artifact rather than to model output.

Statements. Sole author; no competing interests; no external funding. Archived at Zenodo, DOI 10.5281/zenodo.20494383.

7. Conclusion

A full electronic-structure stack runs in a browser tab, validated against PySCF, and a browser-tab swarm scales it to C_{60} without native code, a data-center GPU, or an install. Where it overlaps native suites it is 2–6× slower; its value lies elsewhere — in removing the install, hardware, and cost barriers between a learner and a real calculation, and in showing that the browser’s memory ceiling is not a hard limit but one that a tab swarm can raise. The entire artifact is a URL, and a referee can re-run every number reported here.

References

Citation metadata (authors, venues, years) collected from the sources; page numbers to be finalized against the publisher of record before submission. Entries marked (*software*) are cited by repository/archive.

- [1] X. Wu, Q. Sun, et al. “Enhancing GPU-acceleration in the Python-based Simulations of Chemistry Framework (GPU4PySCF).” *arXiv:2407.09700* (2024).
- [2] M. Manathunga, et al. “QUICK: A GPU-enabled ab initio quantum chemistry software package.” *Software*, github.com/merzlab/QUICK.
- [3] X. Li, et al. “VeloxChem: GPU-Accelerated Fock Matrix Construction.” *Journal of Chemical Theory and Computation* (2025), PMC11744785.
- [4] E. Lavoie and L. Hendren. “Pando: Personal Volunteer Computing in Browsers.” *arXiv:1803.08426* (2018).
- [5] E. Lavoie, L. Hendren, et al. “Genet: A Quickly Scalable Fat-Tree Overlay for Personal Volunteer Computing using WebRTC.” *arXiv:1904.11402* (2019).
- [6] S. R. Wilkinson and J. Almeida. “QMachine: commodity supercomputing in web browsers.” *BMC Bioinformatics* 15:176 (2014), PMC4063228.
- [7] “Characterizing WebGPU Dispatch Overhead for LLM Inference Across Four GPU Vendors, Three Backends, and Three Browsers.” *arXiv:2604.02344* (2026).
- [8] Q. Sun, et al. “PySCF: the Python-based Simulations of Chemistry Framework.” *WIREs Computational Molecular Science* 8:e1340 (2018).
- [9] U. Schollwöck. “The density-matrix renormalization group in the age of matrix product states.” *Annals of Physics* 326(1):96–192 (2011).
- [10] L. Goerigk, et al. “A look at the density functional theory zoo with the advanced GMTKN55 database.” *Physical Chemistry Chemical Physics* 19:32184–32215 (2017).
- [11] NIST Computational Chemistry Comparison and Benchmark Database (CCCBDB), Release 22, NIST Standard Reference Database 101.

*Draft v0.5 — abstract, methods (incl. full swarm protocol §2.4), validation w/ test-coverage, results + **three figures** (scaling, memory-wall, swarm-protocol sequence), honest limitations, software- availability + reproducibility, numbered references. Numbers sourced from committed experiments/results/ JSON and e2e bench logs; Zenodo DOI 10.5281/zenodo.20494383. Remaining before submission: LaTeX conversion (JOSS / SoftwareX / a chem-software track) and finalizing reference page numbers against publishers of record. Content + figures complete.*